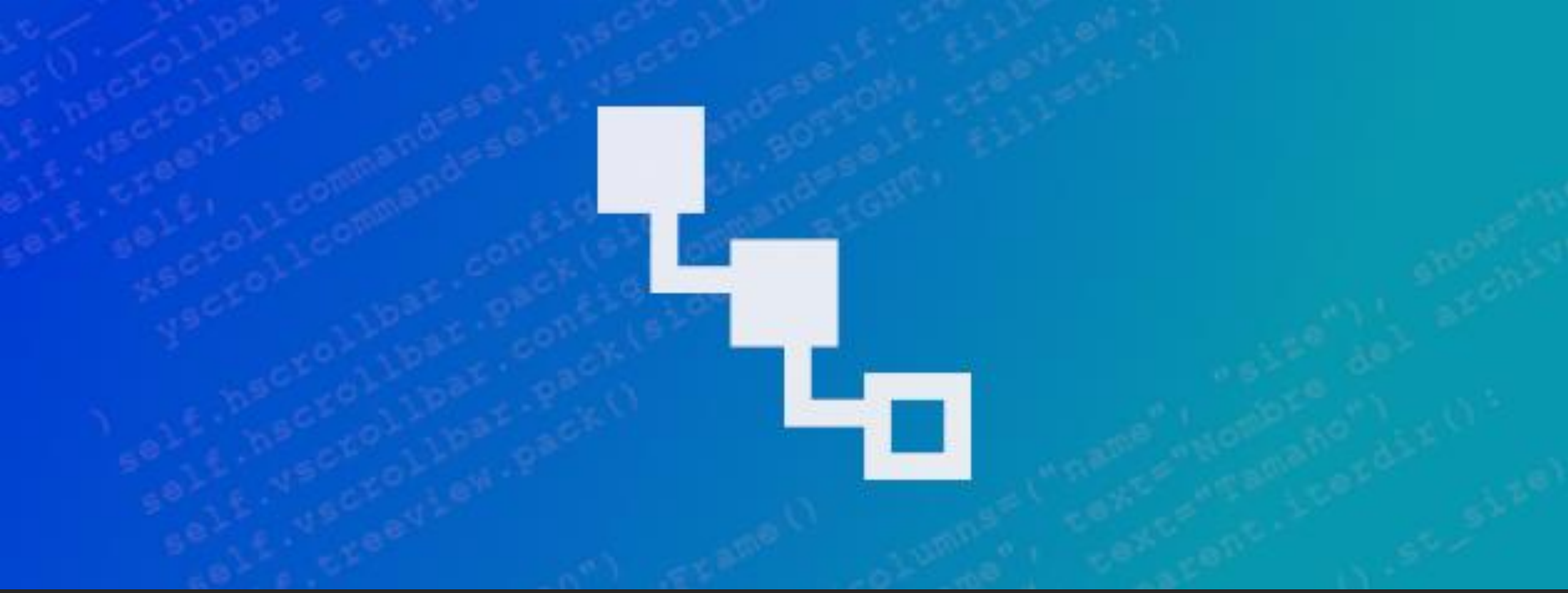


"Dieses Foto" von Unbekannter Autor ist lizenziert gemäß CC BY-NC

Iterator

04.11.2025

Plan für die Woche

- x
- y
- z
- a
- b

Lernziel

- Iteratoren verstehen und anwenden können

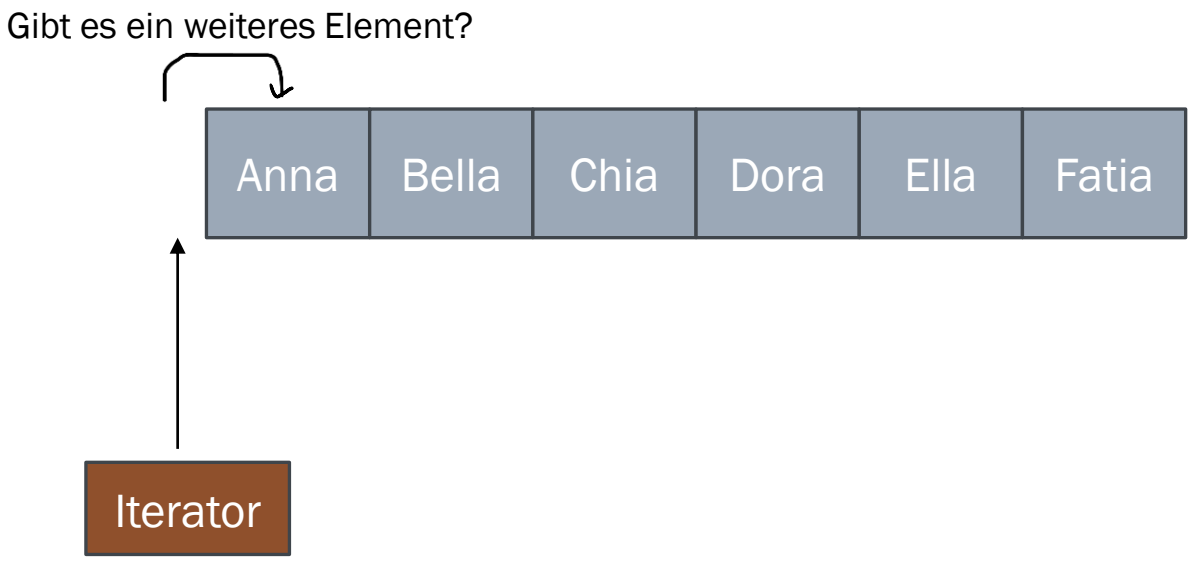
Iterator

Grundlagen

- Schleifen können genutzt werden, um über Collections (ArrayLists, HashSets,....) zu iterieren
- **Problem:**
  - Manche Collections haben keinen Index!
  - Löschen eines Elements innerhalb einer Schleife führt zu Fehlern (ConcurrentModificationException)
- **Lösung:** Iterator

Grundlagen

– Grundprinzip des Iterators:

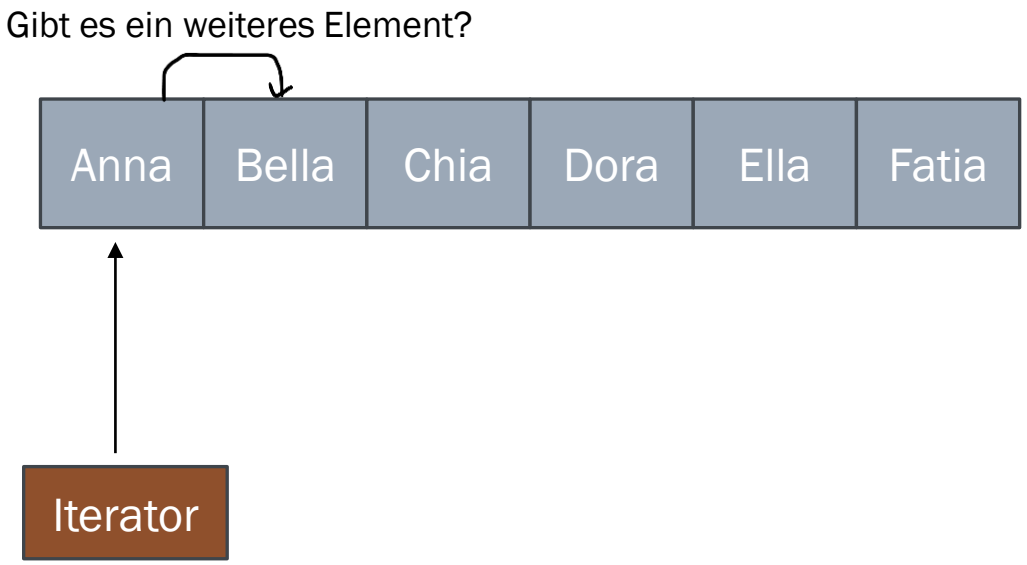


Grundlagen

– Grundprinzip des Iterators:

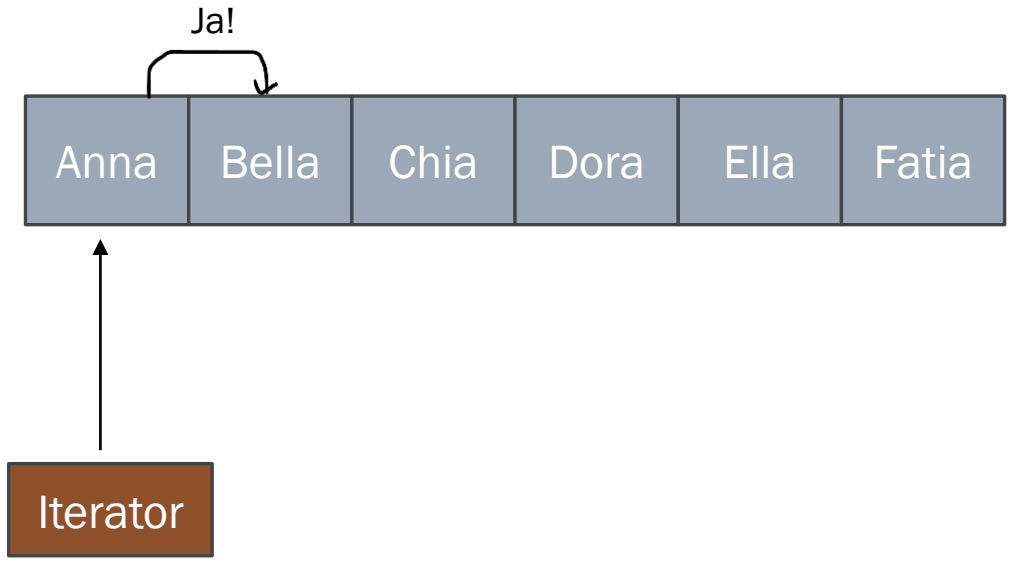
Grundlagen

– Grundprinzip des Iterators:



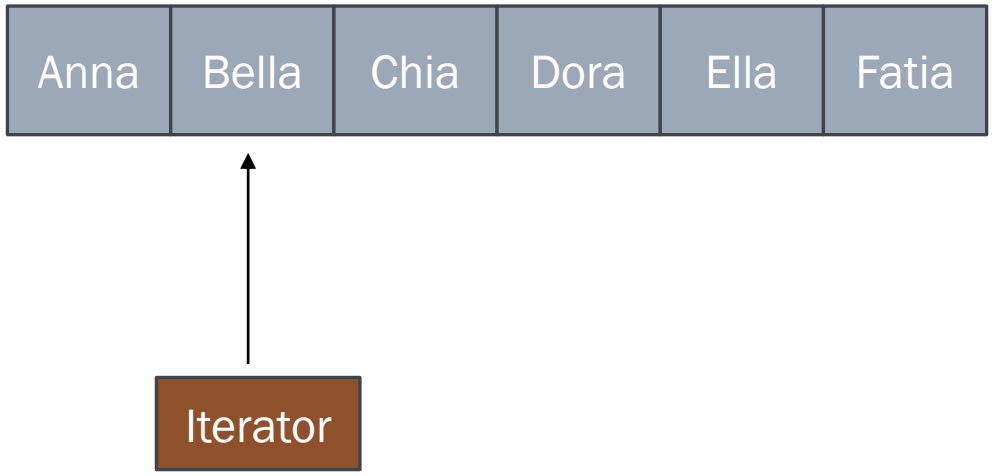
Grundlagen

– Grundprinzip des Iterators:



Grundlagen

– Grundprinzip des Iterators:



Grundlagen

– Grundprinzip des Iterators:

Grundlagen

– Grundprinzip des Iterators:

Grundlagen

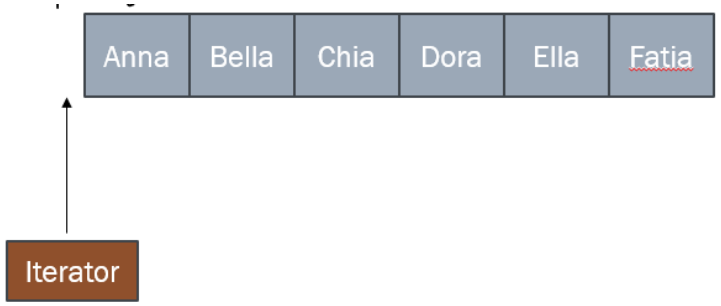
– Grundprinzip des Iterators:

Grundlagen

- Iterator ist ein Interface, das eine Möglichkeit bietet, alle Elemente einer Collection nacheinander zu durchlaufen
- hat folgende Methoden:

Methode	Beschreibung
hasNext()	Prüft, ob noch ein weiteres Element vorhanden ist
next()	Gibt das nächste Element zurück
remove()	Entfernt das zuletzt zurückgegebene Element aus der Collection

Grundlagen



- jede Collection bietet eine Methode iterator(), welche dieses Interface implementiert
- hier muss wie in ArrayLists der Typ angegeben werden
- Syntax:

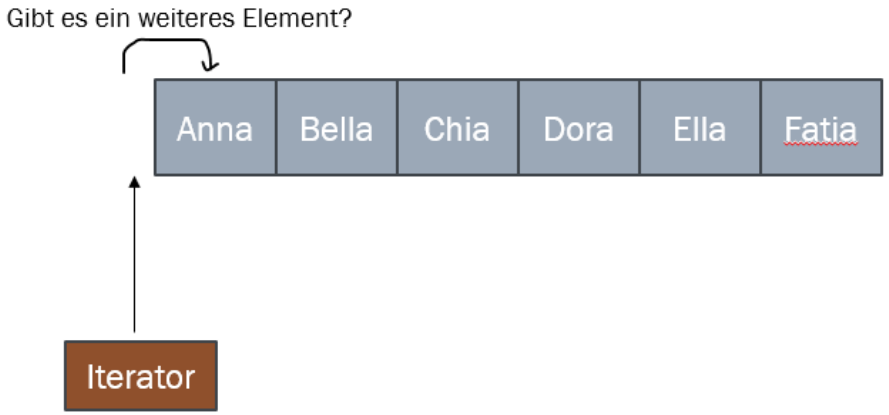
```
ArrayList<String> namen = new ArrayList<>();
namen.add("Anna");
namen.add("Ben");
namen.add("Clara");

Iterator<String> it = namen.iterator();
```

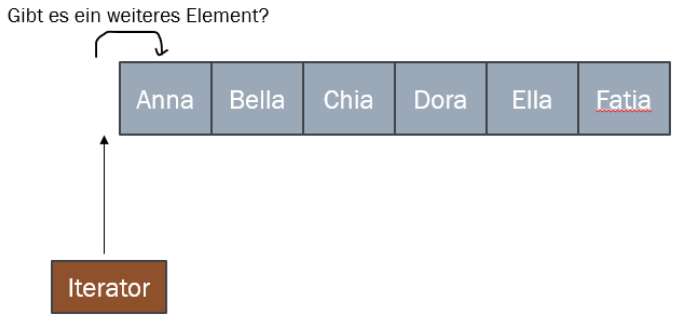
Typ, welchen der Iterator liefern soll

Welche Methode fragt, ob es ein nächstes Element gibt?

Methode	Beschreibung
hasNext()	Prüft, ob noch ein weiteres Element vorhanden ist
next()	Gibt das nächste Element zurück
remove()	Entfernt das zuletzt zurückgegebene Element aus der Collection



Grundlagen



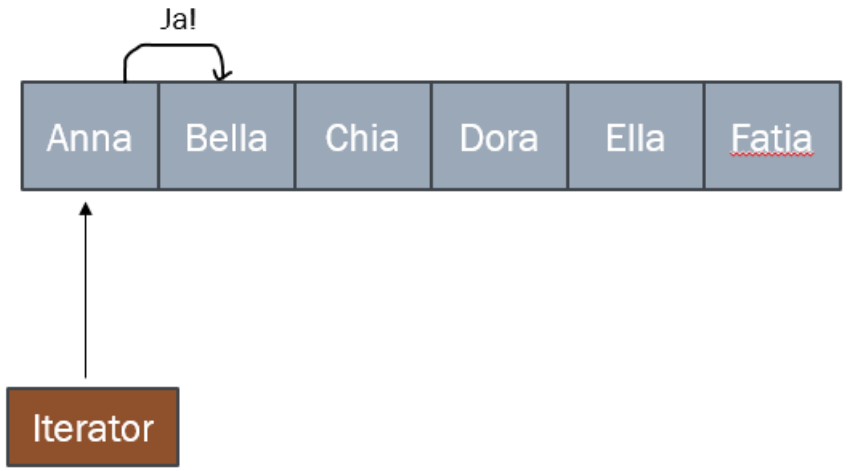
- um von der jetzigen Position des Iterators zu erfahren, ob es ein nächstes Element gibt, muss *hasNext*() aufgerufen werden
- Syntax:

```
ArrayList<String> namen = new ArrayList<>();
namen.add("Anna");
namen.add("Ben");
namen.add("Clara");

Iterator<String> it = namen.iterator();
boolean hasNext = it.hasNext();    //true
```

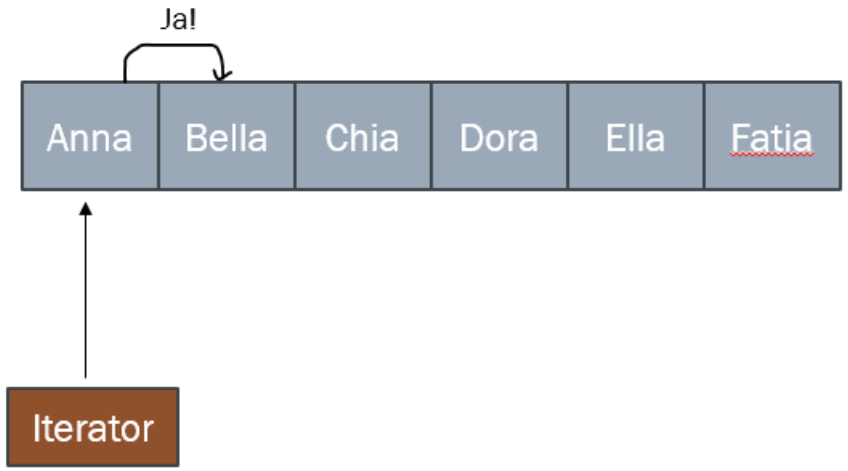
Welche Methode geht in das nächste Element über?

Methode	Beschreibung
hasNext()	Prüft, ob noch ein weiteres Element vorhanden ist
next()	Gibt das nächste Element zurück
remove()	Entfernt das zuletzt zurückgegebene Element aus der Collection

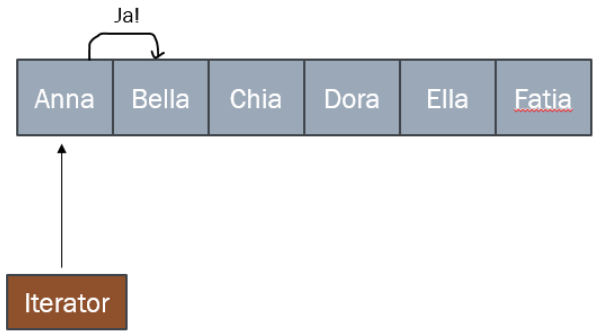


Was muss man beachten, bevor man auf das nächste Element zugreift?

Methode	Beschreibung
hasNext()	Prüft, ob noch ein weiteres Element vorhanden ist
next()	Gibt das nächste Element zurück
remove()	Entfernt das zuletzt zurückgegebene Element aus der Collection



Grundlagen



- um von der Startposition auf die Position in der Collection zuzugreifen, muss *next*() aufgerufen werden
- Syntax:

```
ArrayList<String> namen = new ArrayList<>();
namen.add("Anna");
namen.add("Ben");
namen.add("Clara");

Iterator<String> it = namen.iterator();
if(it.hasNext()) {
    System.out.println(it.next());
}
```

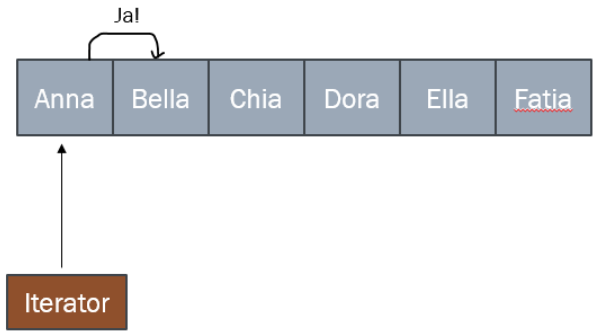
Erst überprüfen, ob es ein nächstes Element gibt

– Wie erreichen wir, dass das für alle Elemente gemacht wird?

```
ArrayList<String> namen = new ArrayList<>();
namen.add("Anna");
namen.add("Ben");
namen.add("Clara");

Iterator<String> it = namen.iterator();
if(it.hasNext()) {
    System.out.println(it.next());
}
```

Grundlagen



- um das für alle Elemente in einer Liste auszuführen, muss man eine Schleife nutzen
- Syntax:

Führt it.next() aus, solange it.hasNext() true.

```
ArrayList<String> namen = new ArrayList<>();
namen.add("Anna");
namen.add("Ben");
namen.add("Clara");

Iterator<String> it = namen.iterator();
while(it.hasNext()) {
    System.out.println(it.next());
}
```

Intern nutzt eine foreach-Schleife auch den Iterator!

Elemente entfernen

- Direkte Entfernung während der Iteration mit einer normalen foreach-Schleife kann Fehler verursachen
- Beispiel:

```
List<String> list = new ArrayList<>();
list.add("a");
list.add("b");
list.add("c");
list.add("d");

for (String elem : list) {
    if (elem.equals("b")) {
        list.remove(elem);
    }
}
```

Elemente entfernen

- remove() – Methode von Iterator löscht Elemente sicher
- Beispiel:

```
List<String> list = new ArrayList<>();
list.add("a");
list.add("b");
list.add("c");
list.add("d");
Iterator<String> iterator = list.iterator();
while(iterator.hasNext()){
    String elem = iterator.next();
    if (elem.equals("b")) {
        iterator.remove();
    }
}
```

Elemente entfernen

– Warum funktioniert folgender Code nicht?

```
List<String> list = new ArrayList<>();
list.add("a");
list.add("b");
list.add("c");
list.add("d");
Iterator<String> iterator = list.iterator();
while(iterator.hasNext()){
    String elem = iterator.next();
    if (elem.equals("b")) {
        list.remove(elem);
    }
}
```

Zusammenfassung

Zusammenfassung

Iterator: Objekt, das alle Elemente einer Collection durchläuft

Methoden: hasNext(), next(), remove()

Vorteile:

- Funktioniert für alle Collections
- Sicheres Entfernen von Elementen während der Iteration

Tipps:

- Generics nutzen
- Immer hasNext() prüfen
- remove() nur über Iterator aufrufen